

Stock Market Portfolio Tracker

A MySQL-based system for tracking stock investments, transactions, and portfolio performance.

Data Analytics Program — Codegnan

Team: M. Pranav, Veera Badri, M. Yashwanth, Firoz

Project Objective

- **Model a real investment portfolio** — Design a relational schema covering users, stocks, transactions, and daily prices.
- **Track buys, sells & holdings** — Every transaction is logged, and current holdings are derived automatically.
- **Analyze performance with SQL** — Use joins, views, window functions, and subqueries to answer real questions.
- **Prepare for BI reporting** — Structure the output so it can feed directly into a Power BI dashboard.

Tech Stack

Tool	Purpose
MySQL 8	Schema design & queries
MySQL Workbench	Development environment

Tool	Purpose
Power BI	Dashboard & visualization
Python (optional)	Synthetic data generation

Database Schema

7 tables linked by foreign keys, plus one derived view:

- **users** – `user_id` (PK), name, email, created_at
- **portfolios** – `portfolio_id` (PK), `user_id` (FK), portfolio_name
- **stocks** – `stock_id` (PK), ticker, company, sector, exchange
- **transactions** – `transaction_id` (PK), `portfolio_id`, `stock_id` (FK), type, qty, price, date
- **watchlist** – `user_id` (FK), `stock_id` (FK)
- **stock_prices** – `price_id` (PK), `stock_id` (FK), OHLC, volume, date
- **dividends** – `dividend_id` (PK), `stock_id` (FK), amount, date
- **transaction_audit** – `audit_id` (PK), `transaction_id`, action, timestamp
- **holdings_view** (view) – derived from `transactions`; calculates live net quantity and average buy price per stock

Entity Relationship Diagram

```

users —< portfolios —< transactions >—
stocks —< stock_prices
|
| |—< dividends
|
| |—< watchlist >— users
|
| |—<
transaction_audit

transactions --(derived)--> holdings_view

```

PK = Primary key · FK = Foreign key · --- = Derived view

SQL Concepts Applied

- **Foreign Keys & Constraints** — Every table links back to `users` or `stocks`, enforcing referential integrity (`CHECK` on quantity > 0, `UNIQUE` on ticker/date).
- **Views** — `holdings_view` computes live portfolio positions from raw transactions, so nothing has to be manually recalculated.
- **Window Functions** — `RANK()` ranks stocks by performance; `LAG()` compares each price to the previous period for % returns.
- **Correlated Subqueries** — Nested `SELECT` s fetch each stock's most recent price for valuation and P&L calculations.
- **Triggers** — An `AFTER INSERT` trigger on `transactions` automatically writes an audit log entry.

Analytical Queries

1. **Portfolio Valuation** — Current market value of every holding using the latest available price.
2. **Unrealized P&L** — Profit/loss on open positions vs. average buy price.
3. **Top Performers** — Stocks ranked by return using `RANK()`.
4. **Sector Allocation** — Portfolio value broken down by sector, as a percentage.
5. **Monthly Returns** — Period-over-period % price change using `LAG()`.
6. **Transaction Audit** — Auto-logged trail of every BUY/SELL via trigger.

Sample Insight: Current Holdings

- **Holdings:** 6 TCS shares, 20 INFY shares currently open (after 1 partial sell).
- **Valuation:** Total current value ≈ Rs. 34,469 across both positions (TCS: Rs. 13,533 · INFY: Rs. 20,936).
- **Observation:** Both positions show an unrealized loss vs. average buy price at the latest snapshot price — flagged by the P&L query for review.

Future Scope

- **More Data** — Add more users, portfolios, and transactions for richer, multi-user analysis.
- **Automation** — Automate daily price inserts instead of manual snapshots (API or scheduled import).

- **Realized P&L** — Add FIFO / weighted-average costing for accurate sell-side profit tracking.
- **Power BI Dashboard** — Connect the schema to Power BI for live valuation and allocation visuals.